

ADR-001: Adoption of a Modular Monolith Architecture

Status: Accepted Date: 2026-09-30 Scope: REMIND MVP

Context

REMIND is an MVP for coordinating emergency reports between users and organizations capable of responding to them. The current domain contains distinct business capabilities such as identity, company management and emergency response.

There is currently no demonstrated requirement for independent deployment, distributed processing, independent scaling or service-level isolation. A distributed architecture would therefore introduce operational and development cost before the problem requires it.

Architectural complexity must be introduced only in response to demonstrated requirements, measurable constraints, or concrete correctness needs.

Decision

REMIND will be implemented as a **Modular Monolith**: one deployable application with explicit boundaries between business capabilities. Source code will follow **Package by Feature** rather than global technical layers.

The initial conceptual module boundaries are **auth**, **user**, **company**, **emergency** and a deliberately small **shared** area. Exact internal layering is not mandatory and may evolve only when a module's complexity justifies it.

Technology baseline

Technology	Decision / role
Java 21	Language and runtime baseline.
Maven	Build and dependency management.
Spring Boot	Application foundation and runtime.
Spring Security	Authentication and authorization integration.
JWT	API authentication token mechanism.
PostgreSQL	Relational persistence.
Flyway	Versioned database schema migrations.
Testcontainers	Real PostgreSQL infrastructure for integration tests.
Package by Feature	Primary source-code organization strategy.

Module boundary rules

- Business capabilities remain cohesive and own their domain behavior.
- A module must not manipulate another module's internal persistence model as a shortcut.
- Cross-module interaction should occur through deliberate application-level entry points or contracts.
- The shared area must contain genuinely shared concepts and must not become a generic dumping ground.
- Sharing one PostgreSQL database does not imply unrestricted coupling between modules.
- Infrastructure concerns must not define domain rules.
- New abstractions must solve an existing problem rather than anticipate hypothetical future requirements.

Database strategy

The MVP will initially use a single PostgreSQL database. Data ownership should remain conceptually aligned with module boundaries. Flyway will version schema evolution; manual, untracked production schema changes are outside the intended workflow.

Evolution toward microservices

Microservice extraction is **not** an MVP objective. Existing module boundaries preserve an evolutionary option, not a promise of future distribution.

A module may be evaluated for extraction only when concrete pressures emerge, such as materially different scaling characteristics, independent deployment needs, operational isolation, independent evolution, or infrastructure requirements specific to that capability. Any extraction requires a new ADR.

Consequences

- Positive: lower operational complexity, one build/deploy unit, simpler local development, straightforward transactions, explicit business boundaries and practical integration testing.
- Negative: modules share one process and deployment lifecycle; process-level failures can affect the whole application; module boundaries depend on engineering discipline.
- Risk: careless cross-module access can degrade the system into a tightly coupled monolith. Code review and tests should protect boundaries as the project grows.

Alternatives considered

Traditional layered monolith: operationally simple, but global controller/service/repository/entity packages make business ownership less explicit and can encourage cross-feature coupling.

Microservices: rejected for the MVP because no current requirement justifies network boundaries, distributed failure modes, data consistency complexity, independent deployments or additional observability/infrastructure.

Mandatory project-wide Clean Architecture: not adopted as a template. Useful principles may be applied where they solve a real problem, but every feature will not be forced into ports, adapters and layers merely for structural symmetry.

Review conditions

This ADR must be revisited when measurable requirements challenge the current architecture, including independent scaling/deployment, isolation requirements, significant coupling, or operational constraints that cannot be adequately addressed inside the modular monolith. Until such evidence exists, the Modular Monolith remains the architectural baseline.